

Programowanie obiektowe i inżynieria oprogramowania

Lab AG-10:
- Klasy abstrakcyjne.

Akademia Górniczo-Hutnicza im. Stanisława Staszica w Krakowie
AGH University of Science and Technology

Prowadzący: dr inż. Jakub Bryła
2025

Na poprzednich zajęciach dostosowano kod klasy *TCandidate* do bycia wzorcem dla klas osobników o **dwóch genach**. Aktualnie zostaną wprowadzone modyfikacje, które zlikwidują to ograniczenie.

Zmiana sposobu zapisu genów
w osobniku.

Dodanie biblioteki vector

Informacja o liczbie genów jako
parametr stały dla klasyGenotyp typu `vector<TParam>`Deklaracja funkcji do
uzupełnienia genotypu.

```
cpp    TParam.h    TCandidate.h    TCandidate_Zad1.h    TPopulation.h    TAlgorithm.h
32024  (Globalny zasięg)
1      #pragma once
2      #include <vector>
3      #include "TParam.h"
4
5      // #define GENS_COUNT 2
6
7      class TCandidate
8      {
9      protected:
10         // TParam genotype[GENS_COUNT] =
11         // {
12         //     TParam("x1", 0, 100, 1),
13         //     TParam("x2", 0, 10, 1)
14         // };
15         int gens_count = 0;
16         std::vector<TParam> genotype;
17
18         double mark;
19
20     public:
21         TCandidate();
22         TCandidate(const TCandidate &original);
23
24         double get_mark() const { return mark; };
25         virtual void rate();
26
27         void info();
28
29     protected:
30         void init_vector();
31         void rand_gens_val();
32         double get_gen_val(int gen_id) const { return genotype[gen_id].get_val(); }
33     };
34
```

```
im.cpp TCandidate.cpp TPopulation.cpp TAlgorithm.cpp TCandidate
G2024
1 #include <iostream>
2 #include <math.h> // pow
3
4 #include "TCandidate.h"
5
6 using namespace std;
7
8 TCandidate::TCandidate()
9 {
10     init_vector();
11     gens_count = genotype.size();
12
13     mark = 0;
14     rand_gens_val();
15 }
16
17 TCandidate::TCandidate(const TCandidate &original)
18 {
19     mark = original.get_mark();
20
21     for (int i = 0; i < gens_count; i++)
22     {
23         double x_start = original.genotype[i].get_x_start();
24         double x_end = original.genotype[i].get_x_end();
25         double dx = original.genotype[i].get_dx();
26         genotype[i].set_range(x_start, x_end, dx);
27
28         double val = original.get_gen_val(i);
29         genotype[i].set_val(val);
30     }
31
32     gens_count = genotype.size();
33 }
34
35 void TCandidate::rate()
36 {
37     double x1 = genotype[0].get_val();
38     double x2 = genotype[1].get_val();
39
40     mark = pow(x1, 2) + x2;
41 }
42
43 void TCandidate::init_vector()
44 {
45     genotype.push_back({ "x1", 0, 100, 1 });
46     genotype.push_back({ "x2", 0, 100, 1 });
47 }
48
```

Wywołanie funkcji do uzupełnienia genotypu

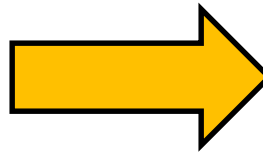
Każde wystąpienie w pliku wyrażenia *GENS_COUNT* należy podmienić na zmienną *gens_count*!

Dwie funkcje charakteryzujące osobnika, tj. rozwiązywany problem:

definicja funkcji oceny → *rate()*,

definicja genotypu → *init_vector()*.

```
main.cpp  TParam.h  TCandidate.h
AG2024
1  #include <iostream>
2  #include <cstdlib> // srand
3  #include <time.h>
4  #include <vector>
5
6  #include "TCandidate.h"
7  #include "TCandidate_Zad1.h"
8  #include "TCandidate_Zad2.h"
9
10 using namespace std;
11
12
13 int main()
14 {
15     srand(time(0));
16
17     TCandidate os{};
18     os.rate();
19     os.info();
20
21     std::cout << "\n\n\n";
22     return 0;
23 }
24
```



```
cs Konsola debugowania programu
=====
==
== gens count: 2
== "x1" value: 12
== "x2" value: 8
==
== rate: 152
=====
```

Klasa abstrakcyjna

Klasa abstrakcyjna

Dwie funkcje charakteryzujące rozwiązywany problem:

- definicja funkcji oceny $\rightarrow rate()$,
- definicja genotypu $\rightarrow init_vector()$.

Przekształcając klasę *TCandidate* na **klasę abstrakcyjną** można wymusić na klasach pochodnych (dziedziczących) konieczność implementacji własnych funkcji *rate()* i *init_vector()*.

W ten sposób kod staje się bardziej odporny na błędy – nie istnieje możliwość zdefiniowania nowego osobnika bez implementacji wspomnianych funkcji – definiujących problem do rozwiązania.

„Wadą” rozwiązania jest brak możliwości utworzenia obiektu klasy *TCandidate*. Klasa ta staje się jedynie „wzorcem” dla klas dziedziczących.

```
h.cpp*  TParam.h  TCandidate.h*  TCandidate_Zad1.h*  TPopulation.h  TAlgorithm.h
G2024   (Globalny zasięg)
1  #pragma once
2  #include <vector>
3  #include "TParam.h"
4
5  class TCandidate
6  {
7  protected:
8      int gens_count = 0;
9      std::vector<TParam> genotype;
10
11      double mark;
12
13  public:
14      TCandidate();
15      TCandidate(const TCandidate &original);
16
17      double get_mark() const { return mark; };
18      virtual void rate() = 0;
19
20      void info();
21
22  protected:
23      virtual void init_vector() = 0;
24      void rand_gens_val();
25      double get_gen_val(int gen_id) const { return genotype[gen_id].get_val(); }
26  };
27
```

Funkcje abstrakcyjne
(linia 18 i 23)
sprawiają, że cała
klasa jest
abstrakcyjna.


```
ram.cpp  TCandidate.cpp*  TPopulation.cpp  TAlgorithm
AG2024  TCandidate
34
35  /*
36  void TCandidate::rate()
37  {
38      double x1 = genotype[0].get_val();
39      double x2 = genotype[1].get_val();
40
41      mark = pow(x1, 2) + x2;
42  }
43
44  void TCandidate::init_vector()
45  {
46      genotype.push_back({ "x1", 0, 100, 1 });
47      genotype.push_back({ "x2", 0, 100, 1 });
48  }
49  */
50
```

Należy usunąć
pierwotny kod funkcji
abstrakcyjnych!

```
8  TCandidate::TCandidate()
9  {
10     // init_vector();
11     gens_count = genotype.size();
12
13     mark = 0;
14     rand_gens_val();
15 }
```

W funkcjach nieabstrakcyjnych
nie może być odwołania do
funkcji abstrakcyjnych!

Klasa abstrakcyjna zawiera funkcje abstrakcyjne, których definicja implementowana jest dopiero w klasach pochodnych. Z tego powodu klasa abstrakcyjna nie może mieć swoich obiektów – służy jedynie jako wzorzec/szkic dla klas pochodnych. Tym samym **funkcje abstrakcyjne** wskazują czym poszczególne klasy pochodne powinny się od siebie różnić.

W naszym przypadku osobnicy różnią się od siebie zakresem wartości dla poszczególnych genów, liczbą genów oraz funkcją oceny.

```
13 int main()
14 {
15     srand(time(0));
16
17     TCandidate os{};
18     os.rate();
19     os.info();
20
21     std::cout <<
22     return 0;
23 }
24
25 /*
26 unsigned int candidates_count = 5;
```

(zmienna lokalna) TCandidate os
Wyszukaj w trybie online

obiekt typu klasy abstrakcyjnej "TCandidate" jest niedozwolony:
funkcja "TCandidate::rate" to czysta funkcja wirtualna
funkcja "TCandidate::init_vector" to czysta funkcja wirtualna
Wyszukaj w trybie online

„klasa abstrakcyjna nie może mieć swoich obiektów”

```
13 int main()
14 {
15     srand(time(0));
16
17     TCandidate_Zad1 os_zad1{};
18     os_zad1.rate();
19     os_zad1.info();
20
21     std::cout << "\n\nr";
22     return 0;
23 }
24
```

(zmienna lokalna) TCandidate_Zad1 os_zad1
Wyszukaj w trybie online

obiekt typu klasy abstrakcyjnej "TCandidate_Zad1" jest niedozwolony:
czysty element wirtualny funkcja "TCandidate::init_vector" nie ma elementu przesłaniającego
Wyszukaj w trybie online

Dla klas dziedziczących należy zdefiniować funkcje abstrakcyjne!

Modyfikacja klas pochodnych

Modyfikacja klas pochodnych



```
cpp TParam.h TCandidate.h TCandidate_Zad1.h TPopulation.h TAlgorithm.h
2024 (Globalny zasięg)
1 #pragma once
2
3 #include "TCandidate.h"
4
5 class TCandidate_Zad1 : public TCandidate
6 {
7 public:
8     TCandidate_Zad1() : TCandidate()
9     {
10         init_vector();
11     }
12
13     TCandidate_Zad1(const TCandidate_Zad1& original) : TCandidate(original) {}
14
15 void rate()
16 {
17     double x1 = genotype[0].get_val();
18     double x2 = genotype[1].get_val();
19
20     mark = 2 * (x1 + x2);
21 }
22
23 protected:
24     void init_vector();
25 };
26
27 void TCandidate_Zad1::init_vector()
28 {
29     genotype.push_back({ "x1", 0, 100, 1 });
30     genotype.push_back({ "x2", 0, 10, 1 });
31
32     gens_count = genotype.size();
33 }
34
35
```

```
n.cpp TCandidate.cpp TPopulation.cpp TAlgorithm.cpp TCandidate_Zad2.h
2024 TCandidate_Zad2
1 #pragma once
2
3 #include "TCandidate.h"
4
5 class TCandidate_Zad2 : public TCandidate
6 {
7 public:
8     TCandidate_Zad2() : TCandidate()
9     {
10         init_vector();
11     }
12
13     TCandidate_Zad2(const TCandidate_Zad2& original) : TCandidate(original) {}
14
15 void rate()
16 {
17     double x1 = genotype[0].get_val();
18     double x2 = genotype[1].get_val();
19
20     mark = 100 * x1 + x2;
21 }
22
23 protected:
24     void init_vector();
25 };
26
27 void TCandidate_Zad2::init_vector()
28 {
29     genotype.push_back({ "x1", 0, 10, 1 });
30     genotype.push_back({ "x2", 11, 20, 2 });
31
32     gens_count = genotype.size();
33 }
```

Do testów należy zakomentować cały kod klas
TPopulation i *TAlgorithm*!

```
cpp TParam.h TCandidate.h TCandidate_Zad1.h TPopulation.h x TAlgorithm.h
2024 (Globalny zasięg)
1 #pragma once
2
3 #include <vector>
4 #include "TCandidate.h"
5
6 /*
7 class TPopulation
8 {
9     static unsigned int    population_count;
10     unsigned int          _id;
11     unsigned int          candidate_count;
12     std::vector<TCandidate> candidates;
13     double                best_val = 0;
14
15 public:
16     TPopulation(unsigned int cand_count = 10);
17     TPopulation(const TPopulation &original);
18     void calculate();
19     TCandidate get_best_candidate();
20
21     unsigned int get_id() { return _id; }
22     unsigned int get_candidates_count() const { return candidate_count; }
23     double get_best_val() const { return best_val; }
24
25     void info();
26
27 private:
28     const TCandidate* get_candidate_wsk(int _id) const;
29 };
30 */
31
```

```
cpp TParam.h TCandidate.h TCandidate_Zad1.h TPopulation.h TAlgorithm.h x
2024 (Globalny zasięg)
1 #pragma once
2
3 /*
4
5 #include "TPopulation.h"
6
7 class TAlgorithm
8 {
9     unsigned int stop_max_population_count;
10     unsigned int stop_min_improvement_proc;
11
12     TPopulation* wsk_population_pres = nullptr;
13     TPopulation* wsk_population_prev = nullptr;
14
15 public:
16     TAlgorithm(unsigned int candidates_count = 10,
17                unsigned int max_population_count = 20,
18                unsigned int min_improvement_proc = 3);
19
20     ~TAlgorithm();
21
22     void run();
23
24 private:
25     bool is_stop();
26     bool is_max_population();
27     bool is_min_improvement();
28 };
29 */
30
```

Również plików *.cpp!

Modyfikacja klas pochodnych - TEST

```
main.cpp  TParam.h  TCandidate.h  TCand
AG2024 (Globaln
1  #include <iostream>
2  #include <cstdlib> // srand
3  #include <time.h>
4  #include <vector>
5
6  #include "TCandidate_Zad1.h"
7  #include "TCandidate_Zad2.h"
8
9  using namespace std;
10
11
12  int main()
13  {
14      srand(time(0));
15
16      TCandidate_Zad1 os_zad1{};
17      os_zad1.rate();
18      os_zad1.info();
19
20      TCandidate_Zad2 os_zad2{};
21      os_zad2.rate();
22      os_zad2.info();
23
24      std::cout << "\n\n\n";
25      return 0;
26  }
27
```



```
Konsola debugowania programu !
=====
==
== gens count: 2
== "x1" value: 4
== "x2" value: 9
==
== rate: 26
=====
==
== gens count: 2
== "x1" value: 8
== "x2" value: 13
==
== rate: 813
=====
```

Utworzenie nowej klasy pochodnej

Utworzenie nowej klasy pochodnej

```
m.cpp  TCandidate_Zad3.h  TCandidate.cpp  TPopulation.cpp  TAlgorithm.cpp  TCanc
2024  (Globalny zasięg)
1  #pragma once
2
3  #include "TCandidate.h"
4
5  class TCandidate_Zad3 : public TCandidate
6  {
7  public:
8      TCandidate_Zad3() : TCandidate()
9      {
10         init_vector();
11     }
12
13     TCandidate_Zad3(const TCandidate_Zad3& original) : TCandidate(original) {}
14
15     void rate()
16     {
17         double x1 = genotype[0].get_val();
18         double x2 = genotype[1].get_val();
19         double x3 = genotype[2].get_val();
20
21         mark = 100 * x1 + 10 * x2 + x3;
22     }
23
24     protected:
25         void init_vector();
26     };
27
28     void TCandidate_Zad3::init_vector()
29     {
30         genotype.push_back({ "x1", 0, 10, 1 });
31         genotype.push_back({ "x2", 0, 10, 1 });
32         genotype.push_back({ "x3", 0, 10, 1 });
33
34         gens_count = genotype.size();
35     }
36
```

Utworzenie nowej klasy pochodnej - TEST

```
main.cpp  TParam.h  TCandidate.h  TCandid
AG2024  (Globalny z
1  #include <iostream>
2  #include <cstdlib> // srand
3  #include <time.h>
4  #include <vector>
5
6  #include "TCandidate_Zad1.h"
7  #include "TCandidate_Zad2.h"
8  #include "TCandidate_Zad3.h"
9
10 using namespace std;
11
12
13 int main()
14 {
15     srand(time(0));
16
17     TCandidate_Zad1 os_zad1{};
18     os_zad1.rate();
19     os_zad1.info();
20
21     TCandidate_Zad2 os_zad2{};
22     os_zad2.rate();
23     os_zad2.info();
24
25     TCandidate_Zad3 os_zad3{};
26     os_zad3.rate();
27     os_zad3.info();
28
29     std::cout << "\n\n\n";
30     return 0;
31 }
```



```
Konsola debugowania programu 1
=====
==
== gens count: 2
== "x1" value: 51
== "x2" value: 9
==
== rate: 120
=====
==
== gens count: 2
== "x1" value: 0
== "x2" value: 17
==
== rate: 17
=====
==
== gens count: 3
== "x1" value: 6
== "x2" value: 8
== "x3" value: 9
==
== rate: 689
=====
```

Rozwiązywanie problemu z klasami *TPopulation* i *TAlgorithm*

```
cpp TParam.h TCandidate.h TCandidate_Zad1.h TPopulation.h* TAAlgorithm.h
32024 (Globalny zasięg)
1  #pragma once
2
3  #include <vector>
4  #include "TCandidate.h"
5
6  class TPopulation
7  {
8      static unsigned int    population_count;
9      unsigned int          _id;
10     unsigned int           candidate_count;
11     std::vector<TCandidate*> candidates;
12     double                 best_val = 0;
13
14 public:
15     TPopulation(unsigned int cands_count = 10);
16     TPopulation(const TPopulation &original);
17     void calculate();
18     TCandidate* get_best_candidate();
19
20     unsigned int get_id() { return _id; }
21     unsigned int get_candidates_count() const { return candidate_count; }
22     double get_best_val() const { return best_val; }
23
24     void info();
25
26 private:
27     const TCandidate* get_candidate_wsk(int _id) const;
28 };
29
```

Praca na wskaźnikach.

Przypomnienie: wskaźnik *TCandidate* może wskazywać na obiekty klas dziedziczących.

Zamiana w każdym miejscu odwołania do obiektu klasy *TCandidate* na wskaźnik do obiektu klasy *TCandidate*.

```
im.cpp  TCandidate_Zad3.h  TCandidate.cpp  TPopulation.cpp*  TAlgorithm.cpp
32024
19
20 TPopulation::TPopulation(const TPopulation& original)
21 {
22     _id = population_count;
23     population_count++;
24
25     candidate_count = original.get_candidates_count();
26     best_val = original.get_best_val();
27
28     for (int i = 0; i < candidate_count; i++)
29     {
30         const TCandidate* wsk_os_org = original.get_candidate_wsk(i);
31         TCandidate copy{*wsk_os_org};
32         candidates.push_back(copy);
33     }
34
35     cout << "liczba osobników: " << candidates.size() << endl;
36 }
37
```

Pozostaje problem z konstruktorem kopiującym klasy *TCandidate*!

Przypomnienie: konstruktory nie są dziedziczone przez klasy pochodne.

Slajdy 22-25 dotyczą naprawy kodu konstruktora kopiującego klasy *TPopulation*. W przypadku braku jego implementacji, wspomniane slajdy można pominąć.

Utworzenie dwóch funkcji abstrakcyjnych do tworzenia obiektów wybranego typu – odpowiedniki konstruktora domyślnego i kopiującego.

Konstruktory nie mogą być wirtualne, a tym bardziej abstrakcyjne.

Funkcja `create_copy()`, odpowiednik konstruktora kopiującego, obiecuje nie zmieniać stanu elementu na rzecz którego jest wywoływana.

```
cpp    TParam.h    TCandidate.h  TCandidate_Zad1.h    TPopulation.h    TAlgorithm.h
2024                                     (Globalny zasięg)
1      #pragma once
2      #include <vector>
3      #include "TParam.h"
4
5      class TCandidate
6      {
7      protected:
8          int gens_count = 0;
9          std::vector<TParam> genotype;
10
11         double mark;
12
13     public:
14         TCandidate();
15         TCandidate(const TCandidate &original);
16
17         virtual TCandidate* create() = 0;
18         virtual TCandidate* create_copy() const = 0;
19
20         double get_mark() const { return mark; };
21         virtual void rate() = 0;
22
23         void info();
24
25     protected:
26         virtual void init_vector() = 0;
27         void rand_gens_val();
28         double get_gen_val(int gen_id) const { return genotype[gen_id].get_val(); }
29     };
30
```

Wywołanie funkcji przez obiekt tej klasy spowoduje dynamiczne utworzenie nowego obiektu z użyciem konstruktora domyślnego.

Wywołanie funkcji przez obiekt tej klasy spowoduje dynamiczne utworzenie nowego obiektu z użyciem konstruktora kopiującego.

```
TCandidate.h  TParam.h  TCandidate.h  TCandidate_Zad1.h  TPopulation.h  TAlgorithm.h
G2024 (Globalny zasięg)
1  #pragma once
2
3  #include "TCandidate.h"
4
5  class TCandidate_Zad1 : public TCandidate
6  {
7  public:
8      TCandidate_Zad1() : TCandidate()
9      {
10         init_vector();
11     }
12
13     TCandidate_Zad1(const TCandidate_Zad1& original) : TCandidate(original) {}
14
15     TCandidate* create()
16     {
17         return new TCandidate_Zad1();
18     }
19
20     TCandidate* create_copy() const
21     {
22         return new TCandidate_Zad1{ *this };
23     }
24 }
```

Funkcje zwracają wskaźnik do utworzonych obiektów!

```
im.cpp  TCandidate.cpp  TPopulation.cpp  TAlgorithm.cpp  TCandidate_Zad2.h  X
52024  TCandidate_Zad2
1  #pragma once
2
3  #include "TCandidate.h"
4
5  class TCandidate_Zad2 : public TCandidate
6  {
7  public:
8      TCandidate_Zad2() : TCandidate()
9      {
10         init_vector();
11     }
12
13     TCandidate_Zad2(const TCandidate_Zad2& original) : TCandidate(original) {}
14
15     TCandidate* create()
16     {
17         return new TCandidate_Zad2();
18     }
19
20     TCandidate* create_copy() const
21     {
22         return new TCandidate_Zad2{ *this };
23     }
```

```
im.cpp  TParam.h  TCandidate.h  TCandidate_Zad1.h  TCandidate_Zad3.h  X  TPopulation.h
G2024  (Globalny zasięg)
1  #pragma once
2
3  #include "TCandidate.h"
4
5  class TCandidate_Zad3 : public TCandidate
6  {
7  public:
8      TCandidate_Zad3() : TCandidate()
9      {
10         init_vector();
11     }
12
13     TCandidate_Zad3(const TCandidate_Zad3& original) : TCandidate(original) {}
14
15     TCandidate* create()
16     {
17         return new TCandidate_Zad3();
18     }
19
20     TCandidate* create_copy() const
21     {
22         return new TCandidate_Zad3{ *this };
23     }
```



```
am.cpp  TCandidate.cpp  TPopulation.cpp  TAlgorithm.cpp  TCandidate_Zad2.h
G2024
19
20 TPopulation::TPopulation(const TPopulation& original)
21 {
22     _id = population_count;
23     population_count++;
24
25     candidate_count = original.get_candidates_count();
26     best_val = original.get_best_val();
27
28     for (int i = 0; i < candidate_count; i++)
29     {
30         const TCandidate* wsk_os_org = original.get_candidate_wsk(i);
31         TCandidate* copy = wsk_os_org->create_copy();
32         candidates.push_back(copy);
33     }
34
35     cout << "liczba osobników: " << candidates.size() << endl;
36 }
37
```

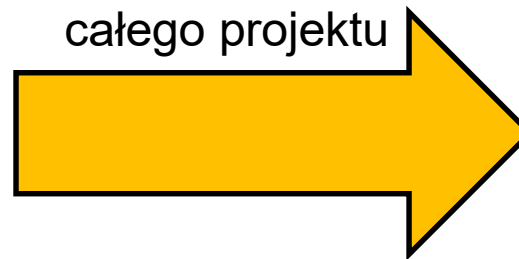
Ostatnie zmiany w klasie *TPopulation*

Kod klasy TAlgorithm wystarczy odkomentować.

```

main.cpp  TParam.h  TCandidate.h  TCar
AG2024
1  #include <iostream>
2  #include <cstdlib> // srand
3  #include <time.h>
4  #include <vector>
5
6  #include "TCandidate_Zad1.h"
7  #include "TCandidate_Zad2.h"
8  #include "TCandidate_Zad3.h"
9
10 using namespace std;
11
12
13 int main()
14 {
15     srand(time(0));
16
17     TCandidate_Zad1 os_zad1{};
18     os_zad1.rate();
19     os_zad1.info();
20
21     TCandidate_Zad2 os_zad2{};
22     os_zad2.rate();
23     os_zad2.info();
24
25     TCandidate_Zad3 os_zad3{};
26     os_zad3.rate();
27     os_zad3.info();
28
29     std::cout << "\n\n\n";
30     return 0;
31 }
    
```

Przywrócenie działania
całego projektu



```

C# Konsola debugowania programu

=====
==
== gens count: 2
== "x1" value: 53
== "x2" value: 2
==
== rate: 110
=====

=====
==
== gens count: 2
== "x1" value: 2
== "x2" value: 15
==
== rate: 215
=====

=====
==
== gens count: 3
== "x1" value: 4
== "x2" value: 3
== "x3" value: 10
==
== rate: 440
=====
    
```

Implementacja nowych funkcjonalności w klasie TPopulation

```
i.cpp  TParam.h  TCandidate.h  TCandidate_Zad1.h  TCandidate_Zad3.h  TPopulation.h  X
G2024  (Globalny zasięg)
1  #pragma once
2
3  #include <vector>
4  #include "TCandidate.h"
5
6  class TPopulation
7  {
8      static unsigned int    population_count;
9      unsigned int          _id;
10     unsigned int           candidate_count;
11     std::vector<TCandidate*> candidates;
12     double                 best_val = 0;
13
14 public:
15     TPopulation(unsigned int cands_count, TCandidate* pattern);
16     TPopulation(const TPopulation &original);
17     void calculate();
18     TCandidate* get_best_candidate();
19
20     unsigned int get_id() { return _id; }
21     unsigned int get_candidates_count() const { return candidate_count; }
22     double get_best_val() const { return best_val; }
23
24     void info();
25
26 private:
27     const TCandidate* get_candidate_wsk(int _id) const;
28 };
29
```

```
im.cpp  TCandidate.cpp  TPopulation.cpp  TAlgorithm.cpp  TCandidate_Zad2.h
32024   (Globalny zasięg)
1  #include <iostream>
2  #include <algorithm> // max
3
4  #include "TPopulation.h"
5
6  using namespace std;
7
8  unsigned int TPopulation::population_count = 0;
9
10 TPopulation::TPopulation(unsigned int cand_count, TCandidate* pattern)
11 {
12     _id = population_count;
13     population_count++;
14
15     candidate_count = cand_count;
16
17     for (int i = 0; i < cand_count; i++) candidates.push_back(pattern->create());
18 }
```

```
main.cpp  TParam.h  TCandidate.h  TCandidate_Zad1.h
AG2024 (Globalny zasięg)
1  #include <iostream>
2  #include <cstdlib> // srand
3  #include <time.h>
4  #include <vector>
5
6  #include "TCandidate_Zad1.h"
7  #include "TCandidate_Zad2.h"
8  #include "TCandidate_Zad3.h"
9  #include "TPopulation.h"
10 using namespace std;
11
12 int main()
13 {
14     srand(time(0));
15
16     TCandidate* pattern;
17     int count = 0;
18     int _type = -1;
19
20     cout << "Który osobnik [1-3]: ";
21     cin >> _type;
22     cout << "Ilu osobnikow utworzyc? ";
23     cin >> count;
24
25     switch (_type)
26     {
27     case 1:
28         pattern = new TCandidate_Zad1{};
29         break;
30     case 2:
31         pattern = new TCandidate_Zad2{};
32         break;
33     case 3:
34         pattern = new TCandidate_Zad3{};
35         break;
36     default:
37         pattern = new TCandidate_Zad1{};
38     }
39
40     TPopulation pop(count, pattern);
41     pop.calculate();
42     pop.info();
43     TCandidate* best = pop.get_best_candidate();
44     best->info();
45
46     std::cout << "\n\n\n";
47     return 0;
48 }
```



Konsola debugowania programu Microsoft Visual Studio

Który osobnik [1-3]: 1
Ilu osobnikow utworzyc? 3

```
===== POPULATION #0 =====
== candidate#0: 62
== candidate#1: 108
== candidate#2: 194
== best val: 194
=====
```

```
=====
==
== gens count: 2
== "x1" value: 90
== "x2" value: 7
==
== rate: 194
=====
```

C# Konsola debugowania programu Microsoft Visual Studio

Który osobnik [1-3]: 2
Ilu osobnikow utworzyc? 3

```
===== POPULATION #0 =====  
== candidate#0: 611  
== candidate#1: 517  
== candidate#2: 819  
== best val: 819
```

```
=====
```



```
==  
== gens count: 2  
== "x1" value: 8  
== "x2" value: 19  
==  
== rate: 819  
=====
```

C# Konsola debugowania programu Microsoft Visual Studio

Który osobnik [1-3]: 3
Ilu osobnikow utworzyc? 3

```
===== POPULATION #0 =====  
== candidate#0: 746  
== candidate#1: 327  
== candidate#2: 124  
== best val: 746
```

```
=====
```



```
==  
== gens count: 3  
== "x1" value: 7  
== "x2" value: 4  
== "x3" value: 6  
==  
== rate: 746  
=====
```

Implementacja nowych funkcjonalności w klasie TAlgorithm


```
1.cpp  TParam.h  TCandidate.h  TPopulation.h  TAlgorithm.h  X
G2024  (Globalny zasięg)
1  #pragma once
2
3  #include "TPopulation.h"
4
5  class TAlgorithm
6  {
7      unsigned int stop_max_population_count;
8      unsigned int stop_min_improvement_proc;
9
10     TCandidate* pattern = nullptr;
11     TPopulation* wsk_population_pres = nullptr;
12     TPopulation* wsk_population_prev = nullptr;
13
14     public:
15     TAlgorithm(TCandidate* pattern,
16               unsigned int candidates_count = 10,
17               unsigned int max_population_count = 20,
18               unsigned int min_improvement_proc = 3);
19
20     ~TAlgorithm();
21
22     void run();
23
24     private:
25     bool is_stop();
26     bool is_max_population();
27     bool is_min_improvement();
28 };
29
```

```
m.cpp  TCandidate.cpp  TPopulation.cpp  TAlgorithm.cpp  TCandidate_Zad2.h
2024  ↓ TAlgorithm
1  #include <iostream>
2  #include <math.h>
3
4  #include "TAlgorithm.h"
5
6  using namespace std;
7
8  TAlgorithm::TAlgorithm(TCandidate* pattern,
9      unsigned int candidates_count,
10     unsigned int max_population_count,
11     unsigned int min_improvement_proc)
12  {
13      this->pattern = pattern;
14      stop_max_population_count = max_population_count;
15      stop_min_improvement_proc = min_improvement_proc;
16
17      wsk_population_pres = new TPopulation{ candidates_count, pattern };
18  }
```

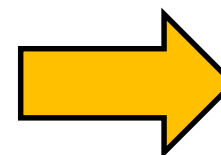
```
am.cpp  TCandidate.cpp  TPopulation.cpp  TAlgorithm.cpp  TCandidate_Zad2.h
G2024   ↓ TAlgorithm   is_stop()

28 void TAlgorithm::run()
29 {
30     bool stop = false;
31
32     while (!wsk_population_prev || !stop)
33     {
34         //(*wsk_population_pres).calculate();
35         wsk_population_pres->calculate();
36
37         cout << "== Population #" << wsk_population_pres->get_id();
38         cout << " || best val: " << wsk_population_pres->get_best_val() << endl;
39
40         stop = is_stop();
41
42         if (!stop)
43         {
44             unsigned int candidates_count = wsk_population_pres->get_candidates_count();
45
46             delete wsk_population_prev;
47             wsk_population_prev = wsk_population_pres;
48
49             // chwilowe rozwiazanie - tworzenie kolejnej losowej populacji
50             wsk_population_pres = new TPopulation{ candidates_count, pattern };
51         }
52     }
53 }
```

```

main.cpp  TPParam.h  TCandidate.h  TPopulation.h
AG2024 (Globalny zasięg)
4  #include <vector>
5
6  #include "TCandidate_Zad1.h"
7  #include "TCandidate_Zad2.h"
8  #include "TCandidate_Zad3.h"
9  #include "TAlgorithm.h"
10 using namespace std;
11
12 int main()
13 {
14     srand(time(0));
15
16     TCandidate* pattern;
17     int count = 0;
18     int _type = -1;
19
20     cout << "Który osobnik [1-3]: ";
21     cin >> _type;
22     cout << "Ilu osobników w populacji? ";
23     cin >> count;
24
25     switch (_type)
26     {
27     case 1:
28         pattern = new TCandidate_Zad1{};
29         break;
30     case 2:
31         pattern = new TCandidate_Zad2{};
32         break;
33     case 3:
34         pattern = new TCandidate_Zad3{};
35         break;
36     default:
37         pattern = new TCandidate_Zad1{};
38     }
39
40     unsigned int candidates_count = 5;
41     unsigned int max_population_count = 20;
42     unsigned int min_improvement_proc = 2;
43
44     TAlgorithm task{ pattern,
45                     candidates_count,
46                     max_population_count,
47                     min_improvement_proc };
48     task.run();
49
50     std::cout << "\n\n\n";
51     return 0;
52 }

```



```

Konsola debugowania programu Microsoft Visual Studio

Który osobnik [1-3]: 3
Ilu osobników w populacji? 10
== Population #0 | best val: 633
== Population #1 | best val: 791
== Population #2 | best val: 1080
== Population #3 | best val: 911
== Population #4 | best val: 869
== Population #5 | best val: 1015
== Population #6 | best val: 1073
== Population #7 | best val: 948
== Population #8 | best val: 1012
== Population #9 | best val: 1088
== Population #10 | best val: 1047
== Population #11 | best val: 1053
== Population #12 | best val: 913
== Population #13 | best val: 1108
== Population #14 | best val: 969
== Population #15 | best val: 1079
== Population #16 | best val: 979
== Population #17 | best val: 1010
== Population #18 | best val: 1001
== Population #19 | best val: 973
== Population #20 | best val: 1021

```